

Machine Learning for Business
Siddharth Ganesh

1.

a)

```
{'metric': 'euclidean', 'n_neighbors': 16}  
0.37790653453856704  
{'metric': 'manhattan', 'n_neighbors': 16}  
0.4289353166666866  
{'metric': 'hamming', 'n_neighbors': 16}  
0.48499143129834044
```

b)

Hamming performed the best of the three metrics. This is because the hamming algorithm is better at calculating distances in multidimensional spaces where there are multiple features not in a continuous space (like the difference between two strings). In this scenario, you end up with more discrete values which is better for Hamming, compared to if you were comparing the relationship between two variables that have more continuous values, Euclidean and Manhattan would be better.

c)

```
{'metric': 'euclidean', 'n_neighbors': 8}  
0.6262362256061544  
{'metric': 'manhattan', 'n_neighbors': 8}  
0.5866742727546432  
{'metric': 'hamming', 'n_neighbors': 16}  
0.48499143129834044
```

Normalized Euclidean performed the best. .63

d)

The result for the hamming metric did not change after we normalized the data. This is because the hamming metric already works the same with or without scaling.

e)

Out of sample

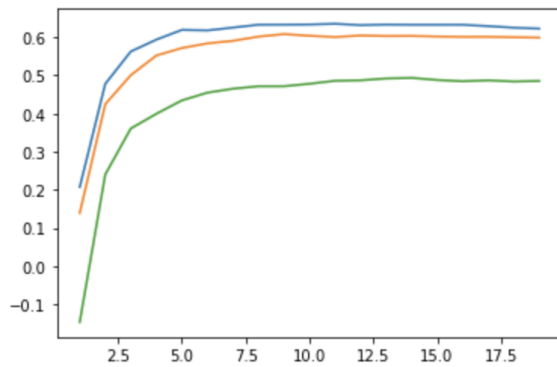
Blue: Euclidean

Orange: Manhattan

Green: Hamming

```
{'metric': 'euclidean', 'n_neighbors': 11}  
0.6406201627179072  
{'metric': 'manhattan', 'n_neighbors': 9}  
0.5827192834109964  
{'metric': 'hamming', 'n_neighbors': 14}  
0.48361426610690783
```

[<matplotlib.lines.Line2D at 0x7fcfaf1d5eb0>]



In sample

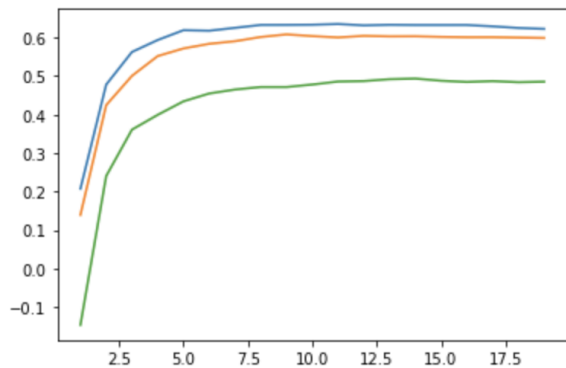
Blue: Euclidean

Orange: Manhattan

Green: Hamming

```
{'metric': 'euclidean', 'n_neighbors': 11}  
0.7085952525122197  
{'metric': 'manhattan', 'n_neighbors': 9}  
0.6917031985280946  
{'metric': 'hamming', 'n_neighbors': 14}  
0.5589751662477733
```

[<matplotlib.lines.Line2D at 0x7fcfae7912b0>]

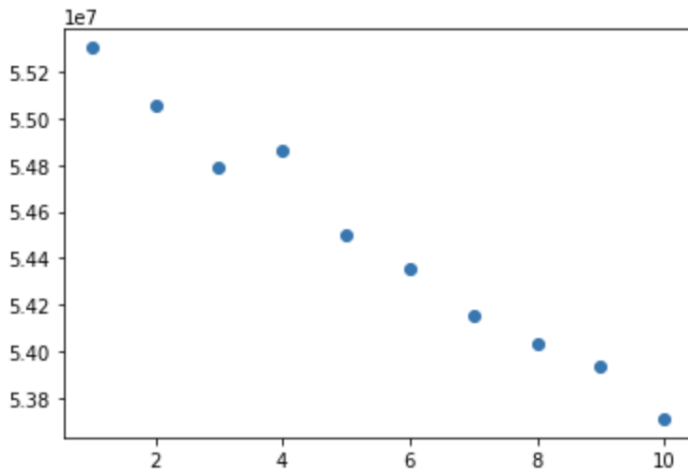
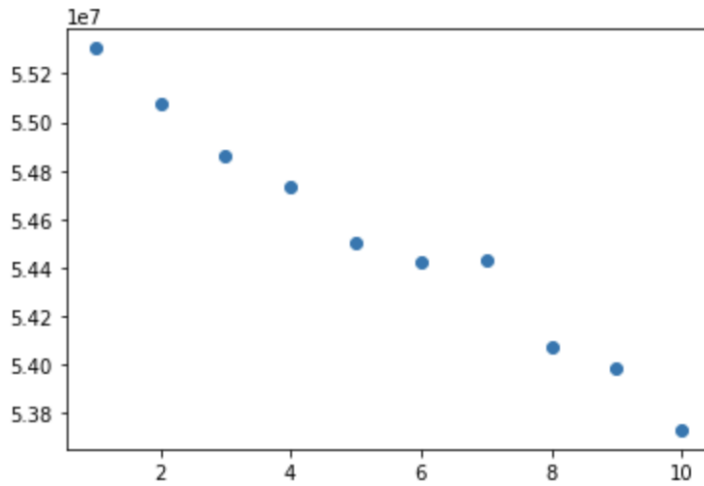


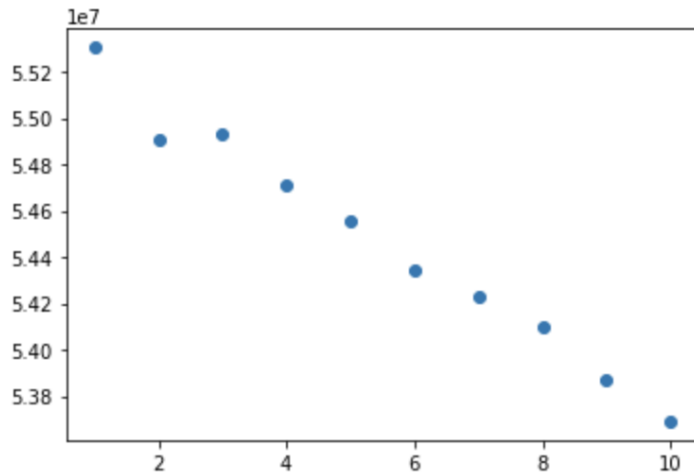
f)

We overfit more as we move to the left of the x-axis because it includes less “neighbors”, so it’s more fitted to individual points instead of being more generalized. Moving to the right would then represent underfitting because we are being much more general and broader in our selection.

2.

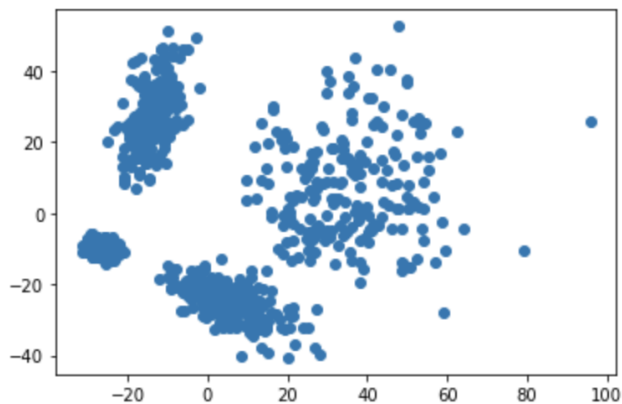
a)





b)
Points in high dimensional spaces are basically the same distance apart so it can be hard to figure out how many clusters to make. We may want to try PCA to reduce the dimensionality.

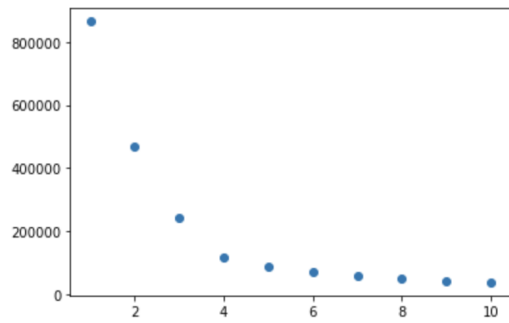
c)
<matplotlib.collections.PathCollection at 0x7fb10f436f10>



d)
The plot seems to indicate **4 clusters**.

e)

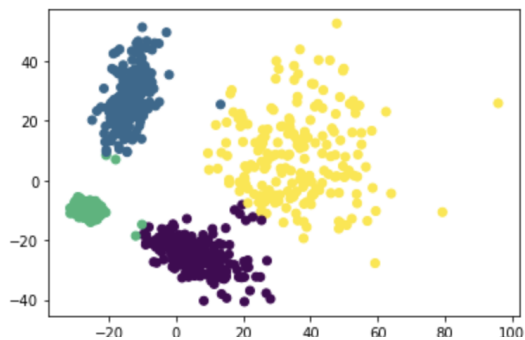
```
<matplotlib.collections.PathCollection at 0x7fb10f1c1730>
```



Yes, the scree plot does confirm our assumption. It seems like the elbow is at 4 clusters.

f)

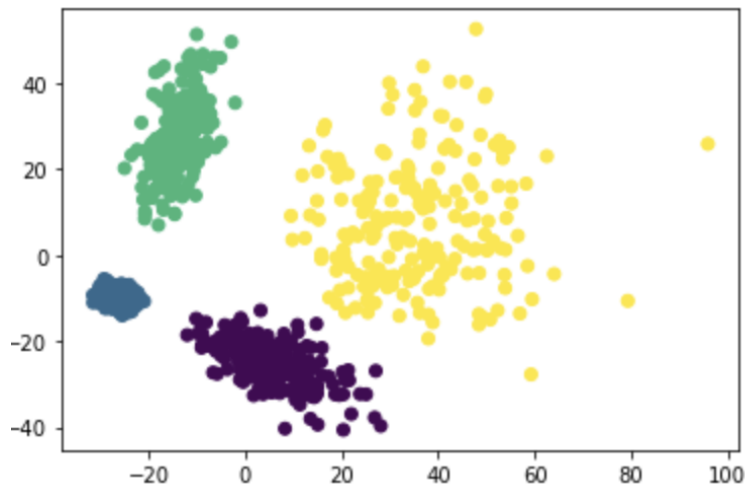
```
<matplotlib.collections.PathCollection at 0x7fb111076370>
```



It seems pretty close to perfect. There are a couple of green dots in, what should be entirely, the purple cluster and blue cluster. There are also some purple dots that should be yellow and one blue dot that should be yellow.

g)

```
clustering = SpectralClustering(n_clusters=4,affinity='precomputed')
knn = KNeighborsTransformer(n_neighbors=150,metric='euclidean')
S = knn.fit_transform(X_train_pca)
clustering.fit(S)
plt.scatter(first,second,c=clustering.labels_)
```



I got this with 150 nearest neighbors.

3.

a)

$w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_1 + w_5x_2 + w_6x_3 + w_7x_1 + w_8x_2 + w_9x_3$ -> The W s need to be tuned (10 parameters)

Explanation: 2 variables go into the top linear node and 1 comes out -> 3 parameters

1 variable goes into the bottom linear node and one comes out 1 -> 2 parameters.

Then the 2 linear nodes go into 1 linear node -> 3 parameters. Then that linear node goes into another linear node -> 2 parameters.

b)

If the filter was a,b,c,d

The function would be $ax_1 + bx_2 + cx_4 + dx_5, ax_2 + bx_3 + cx_5 + dx_6$. A,b,c,d need to be tuned

c)

```
model = nn.Sequential(nn.Flatten(),
                      nn.Linear(28*28,500),
                      nn.ReLU(),
                      nn.Linear(500,250),
                      nn.ReLU(),
                      nn.Linear(250,26))

loss_function = nn.CrossEntropyLoss()

optimizer = Adam(model.parameters())
```

In-sample cross-entropy: 0.1389680802822113

Out-of-sample accuracy: 0.8909

d)

$((6-2 + (2*0))/1) + 1 = 5$ parameters need to be tuned

$785*500 + 501*250 + 251*26 = 524276$ parameters

e)

```
model = nn.Sequential(nn.Conv2d(1,20,5,padding=2),
                      nn.ReLU(),
                      nn.MaxPool2d(2),
                      nn.Conv2d(20,20,5,padding=2),
                      nn.ReLU(),
                      nn.MaxPool2d(2),
                      nn.Flatten(),
                      nn.Linear(20*7*7,100),
                      nn.ReLU(),
                      nn.Linear(100,26))

loss_function = nn.CrossEntropyLoss()

optimizer = Adam(model.parameters())
```

In-sample cross-entropy: 0.22300364077091217

Out-of-sample accuracy: 0.9153

I actually got about 93.5 when I ran it previously, but my kernel crashed because I ran out of “application” memory and I wasn’t able to train past 25 steps after that.

f)

```
model = nn.Sequential(nn.Conv2d(1,32,5,padding=2),
                      nn.ReLU(),
                      nn.MaxPool2d(2),
                      nn.Conv2d(32,64,5,padding=2),
                      nn.ReLU(),
                      nn.MaxPool2d(2),
                      nn.Conv2d(64,128,5,padding=2),
                      nn.ReLU(),
                      nn.MaxPool2d(2),
                      nn.Flatten(),
                      nn.Linear(128*3*3,100),
                      nn.ReLU(),
                      nn.Linear(100,26))

loss_function = nn.CrossEntropyLoss()

optimizer = Adam(model.parameters())
```

In-sample cross-entropy: 0.07180958986282349

Out-of-sample accuracy: 0.9507

4)

- a) My least favorite topic was ridge and lasso regression.
- b) My favorite topic was neural networks.
- c) 2. Good balance.